

UNIVERSITÀ DEGLI STUDI

sbergz on the sburgz

Relatore: Pierpaolo Loreti

Correlatore: Ing. Francesco Mancini

Candidato:

Davide Luci

Anno Accademico 2023-2024

Indice

1	Introduzione	1
1.1	Introduzione al Problema	1
1.2	Oracoli e Meccanismi di Verifica	1
1.2.1	Verifica Statica	2
1.2.2	Verifica Dinamica tramite sistema esterno	3
1.2.3	Verifica Dinamica tramite injection dei comandi di verifica	3
1.2.4	Clonazione del sistema in una rete separata	4
1.3	Cyber-Range: lo stato dell'arte	4
1.3.1	CR Alternativi: kCTF	5
1.3.2	CR Alternativi: CTFBox	8
1.3.3	CR Alternativi: CyTrONE	9
	Elenco delle figure	14
	Elenco dei codici	14
	Elenco degli algoritmi	15

Capitolo 1

Introduzione

1.1 Introduzione al Problema

Nel lavoro di tesi, vogliamo gestire una competizione CTF.

Di solito, in una competizione CTF, uno o piu' utenti (eventualmente organizzati in team) si sfidano per risolvere un'insieme di challenge. Generalmente una challenge vuol dire: recuperare una **flag**.

Ci sono vari tipi di competizioni CTF, tra cui:

- **Jeopardy**: gli utenti devono risolvere vari problemi, la cui flag e' di solito una stringa del tipo `{hai_trovato_la_flag}`
- **Attacco e Difesa**: e' un tipo di competizione dove ogni team, deve recuperare una o piu flag attaccando i sistemi dei team avversare, mentre ognuno si occupa di difendere il proprio sistema.

In particolare, con il nostro lavoro, vogliamo gestire uno o piu' sistemi vulnerabili ad un qualche tipo di attacco.

La flag si ottiene difendendo il sistema, ossia "*patchando*" la vulnerabilita. Dunque la flag non e' piu una stringa come nelle **Jeopardy**, ma piuttosto e': uno stato che deve raggiungere il sistema target mediante il susseguirsi di azioni che il partecipante applica quest'ultimo.

Nel lavoro si tiene conte del fatto che il sistema vulnerabile, spesso, potrebbe essere costituito da uno o piu host/nodi che comunicano tra di loro. Per esempio si condideri il seguente scenario: un server vulnerabile ha bisogno di comunicare con database in esecuzione su un nodo differente da quello del server.

1.2 Oracoli e Meccanismi di Verifica

Per decidere se un team ha recuperato la **flag**, bisogna definire un'**oracolo**, che determini lo stato in cui si trova il sistema target.

L'oracolo e' in grado di accedere al sistema e di fare due azioni:

- puo' leggere la memoria fisica di ogni nodo
- puo' eseguire comandi su qualsiasi nodo

Per ogni tipo di azione eseguita, l'oracolo determina lo stato del sistema confrontando il risultato dell'azione con il valore che l'oracolo si aspettava (ossia i valori vengono validati).

Dunque all'interno del processo di verifica, il sistema si puo' definire *patchato* se e soltanto se tutte le azioni sono validate rispetto al valore che ci si aspettava in output.

Di seguito sono discussi alcuni meccanismi di verifica che abbiamo identificato.

1.2.1 Verifica Statica

La verifica statica prevede che l'oracolo possa accedere alla memoria fisica, comparando le patch applicate nei file in memoria, ad una patch considerata di riferimento per l'oracolo.

Questo tipo di verifica puo' essere implementato in vari modi:

- **diff**: ad ogni file interessato corrisponde un file di riferimento per l'oracolo. Ogni file viene dunque confrontato, riga per riga, con il suo riferimento per determinare se il team ha applicato la patch correttamente.
- **SAST**: Static Analysis Security Testing. Prevede l'accesso al codice sorgente non compilato in modo da analizzare pattern di codice vulnerabili e fallace logiche.¹

Tuttavia, SAST non e' un metodo di analisi efficace, sebbene efficiente:

- possono essere identificati nel processo di verifica, vari falsi positivi (e falsi negativi)¹
- non posso identificare staticamente tutti i tipi di errori di sicurezza ¹
- non posso identificare vulnerabilita all'interno dei binari compilati¹

Dunque l'analisi statica puo' in alcuni casi risultare inesatta, e non coprire tutti i casi d'uso per quanto riguarda la gestione di competizioni CTF.

¹<https://aetic.theiaer.org/archive/v4/v4n3/p1.html> 2. Current Status of SAST based on Checkmarx

1.2.2 Verifica Dinamica tramite sistema esterno

Questo meccanismo di verifica prevede l'uso di un **orchestratore esterno** alle macchine vulnerabili, ossia l'attaccante, che possa interagire con la vulnerabilità utilizzando la rete locale. L'attaccante orchestra l'esecuzione di uno o più comandi, con l'obiettivo di inviare il payload all'interno di un pacchetto di rete.

Durante una competizione di questo tipo, tuttavia, i membri dei team sono legittimati ad accedere alla macchina vulnerabile come utente `root`: così da permettere l'applicazione delle patch sui file di sistema. Questo implica che i partecipanti dei team siano in grado di eseguire tool come `tcpdump` e `wireshark` per intercettare il traffico di rete e di conseguenza, di visualizzarne il payload, ottenendo un suggerimento per la soluzione della challenge.

Inoltre, il payload inviato potrebbe essere tale da mandare in crash il servizio target, causando una penalità alla percentuale di **SLA** (*Service Level Agreement*), ed interferendo potenzialmente con il lavoro del team sfidante (portando per esempio alla perdita di alcuni file). Si vuole dunque trovare un sistema di verifica che esegua completamente all'oscuro del team sfidante.

Il sistema di verifica tramite sistema esterno non copre nemmeno tutte le vulnerabilità. Banalmente, non tutti i servizi vulnerabili sono esposti in rete, ma sono accessibili solo in locale.

Riassumendo quindi, abbiamo identificato alcuni spunti per ideare un meccanismo di verifica migliore:

- Il traffico di rete non dovrebbe essere letto dal team sfidante al fine di ottenere suggerimenti per la soluzione.
- Se l'exploit manda in crash il servizio, questo non dovrebbe penalizzare il team, o più in generale: il team sfidante non vede il servizio distruggersi.
- Dobbiamo gestire anche vulnerabilità non esposte in rete.

1.2.3 Verifica Dinamica tramite injection dei comandi di verifica

Questo meccanismo di verifica prevede l'uso di un **orchestratore esterno** al sistema vulnerabile, che ha la facoltà di eseguire comandi all'interno delle macchine vulnerabili che compongono il sistema. Dunque l'orchestratore esegue uno o più comandi con il fine di verificare che il sistema sia resistente alla vulnerabilità.

Considerando il caso in cui una macchina A contenga un file binario vulnerabile ad un *buffer overflow*, l'orchestratore potrebbe eseguire ad esempio il comando `/path/to/service_logger payload` e verificare l'esito dell'esecuzione.

L'esecuzione di questi comandi è possibile per esempio in ambienti containerizzati, sfruttando utility come `docker exec` o `incus exec`, che permettono di iniettare

comandi all'interno dei container.

Tuttavia: comandi come `pspy`, (che ascoltano attivamente le chiamate di sistema effettuate e gli eventi sul file system in `/proc`) possono intercettare l'esecuzione dei comandi dell'orchestratore e di mostrarli al team, suggerendo al team sfidante la soluzione.²

Dunque, dobbiamo ideare un meccanismo di verifica che deve sia completamente invisibile a questi meccanismi di intercettazione.

1.2.4 Clonazione del sistema in una rete separata

In base a quanto visto prima, in BlueAgent faremo così:

1. Si clonano tutte le macchine del sistema vulnerabile, in una rete separata, mantenendone la topologia originale.
 - – La rete di verifica per il team k -esimo sarà chiamata `verify- $k$`
2. Gli sfidanti non devono poter instradare i pacchetti da e verso la rete `verify- $k$` .
3. Gli sfidanti non devono poter fare injection dei comandi nel sistema nella rete `verify- $k$` .
4. Si aspetta che il sistema si operativi, facendo dei controlli di readiness.
5. Si esegue una batteria di test sul sistema clonato. Questo test è costituita da una serie di comandi da iniettare nel sistema clonato.
6. Per ogni comando eseguito si comparano `stdout`, `stderr` ed `exit_code` con il valore atteso.
7. Se il test ha avuto successo, allora BlueAgent risponde con lo stato `PATCHED`. (TODO: CONTROLLA CHE SIA CORRETTO!)

1.3 Cyber-Range: lo stato dell'arte

Secondo il NIST, un **Cyber-Range** (CR) è definito nel seguente modo:

"Un'ambiente interattivo, che simula la rete locale, i sistemi, i tool e le applicazioni proprietarie di un'azienda o di un'organizzazione".³

Dunque i CR giocano un ruolo fondamentale per formare ed addestrare i futuri professionisti del campo ICT.

Illustreremo nelle sezioni successive alcuni Cyber Range esistenti e valuteremo la loro applicabilità al problema delle competizioni CTF gestite da BlueAgent e se esistono meccanismi di verifica alternativi validi per il nostro scopo.

²<https://github.com/DominicBreuker/pspy>

³<https://www.mdpi.com/1424-8220/20/24/7148> 1. Introduction

1.3.1 CR Alternativi: kCTF

kCTF⁴ e' una piattaforma sviluppata da Google, basata su *Kubernetes* e *nsjail*.

Kubernetes: e' un sistema open source per automatizzare il deploy, scalare e gestire applicazioni containerizzate.⁵

NSJail: ossia, NameSpace Jail, e' un tool Linux per isolare i processi utilizzando varie feature del Kernel Linux come: namespaces, resource limits, seccomp-bpf syscall filters. L'obiettivo di NSJail e' quello di creare un'ambiente sicuro e isolato per eseguire le applicazioni.⁶

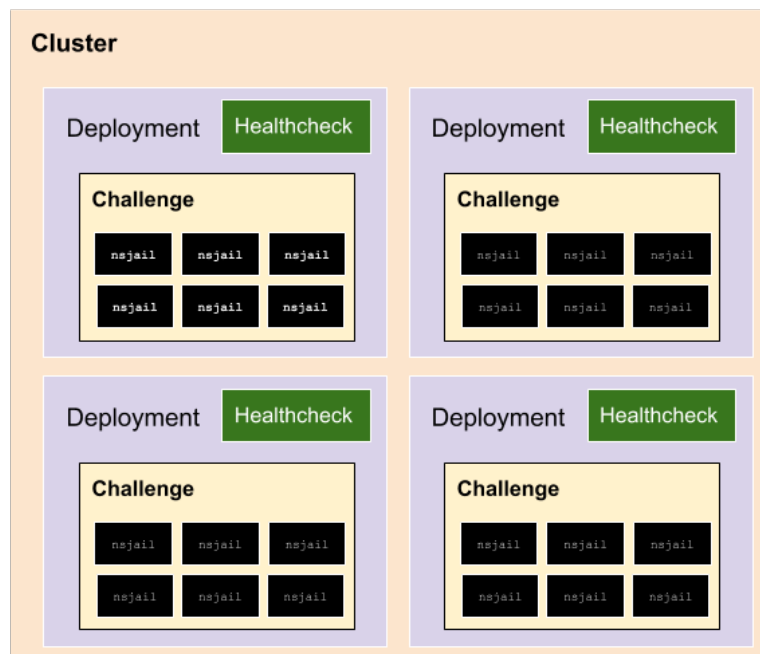


Figura 1.1: Kctf Architecture

Facendo riferimento all'immagine sopra, l'architettura di kCTF prevede:

- Un **cluster**, che contiene un'insieme di challenge
- Dei **deployment**, ognuno contiene due container (o meglio, un Pod che istanzia 2 container):
 - un container **healthcheck**: usato per determinare lo stato di salute dei servizi

⁴<https://google.github.io/kctf/>

⁵<https://kubernetes.io/>

⁶<https://github.com/google/nsjail/>

- un container **challenge**: contiene i servizi su cui lavora lo sfidante.

Pod: in Kubernetes, e' la piu piccola unita che Kubernetes puo' eseguire. Un Pod, rappresenta uno o piu container in esecuzione su un singolo nodo.⁷

Deployment: in Kubernetes, un deployment e' un template che specifica come creare i container. I template guidano la creazione di **Pods** e la gestione sicura del ciclo di vita di un Pod, in particolare la gestione dei ReplicaSet.⁷

NSJail viene avviato su ogni container **challenge**, ed e' configurato per ascoltare su una porta TCP nota. Ad ogni connessione TCP in arrivo, NSJail esegue un fork restituendo allo sfidante un'ambiente separato (rispetto agli altri sfidanti) su cui lavorare. L'obiettivo di questa architettura e' di poter riutilizzare lo stesso container per piu sfidanti.

L'isolamento fornito da NSJail, si ottiene combinando varie tecniche:

- Creazione di nuovi namespace: l'ambiente fornito, esegue su un nuovo namespace, separando logicamente le risorse accessibili.
- Filtraggio delle chiamate di sistema tramite **seccomp-bpf**: questo sottosistema permette di uccidere i processi nell'ambiente se vengono eseguite chiamate di sistema vietate.
- Accounting e limitazione delle risorse con **cgroup**: questo sottosistema permette di monitorare e limitare le risorse utilizzabili nel nuovo ambiente.

NsJail puo' essere configurato in modo che ogni ambiente isolato, abbia a disposizione un'filesystem temporaneo che risiede in RAM, mentre il filesystem originale e' accessibile dall'ambiente sandbox in sola lettura. Questa configurazione e' necessaria poiche altrimenti gli sfidanti modificherebbero tutti lo stesso filesystem.

Un'altra configurazione di NSJail e' la seguente: che e' ideale per le challenge di tipo **pwn** (trovare e sfruttare vulnerabilita in file binari):

```
1 [...]
2 mount: [
3   {
4     src: "/chroot"
5     dst: "/"
6     is_bind: true
7   },
8   {
9     dst: "/tmp"
10    fstype: "tmpfs"
```

⁷<https://www.k8s.guide/workloads/pods-deployments/>

```
11     rw: true
12 },
13 {
14     dst: "/proc"
15     fstype: "proc"
16     rw: true
17 },
18 {
19     src: "/etc/resolv.conf"
20     dst: "/etc/resolv.conf"
21     is_bind: true
22 }
23 ]
```

1. La cartella `/chroot` che risiede nel container, diventa la radice nel nuovo ambiente.
2. Viene creato un nuovo `tmpfs`, in `/tmp`
3. Viene importata la configurazione DNS da `/etc/resolv.conf` dentro la nuova radice.

Tutta via, l'uso di NsJail per separare gli ambienti dei partecipanti, rende difficile l'implementazione di challenge in cui si vuole dare la libertà ai partecipanti di installare pacchetti dentro al container, oppure di modificare file di configurazione che risiedono per esempio in `/etc` e di modificare direttamente servizi in esecuzione sul container. ⁸

Queste limitazioni ci sono perché NsJail nell'ambiente `sandbox` non fornisce una propria istanza di `systemd` (o un qualsiasi gestore di servizi) e neanche l'infrastruttura necessaria per eseguire comandi come `apt`.

Tale limitazione, porta ad escludere kCTF come possibile soluzione al problema originale esposto, suggerendo come astrazione migliore per l'ambiente di un partecipante l'uso di un **container** (Incus o Docker), piuttosto che una `sandbox` generata da NsJail.

TODO: Problemi:

- in nsjail il filesystem è disponibile in sola lettura. Comandi come `apt` e modifiche dirette a `/etc` non funzionano.
- se la connessione TCP cade allora tutto il lavoro che risiede in RAM è perso
- in nsjail, l'applicazione di filtri `seccomp` potrebbero impedire l'esecuzione di alcune chiamate di sistema.
- kCFT usa Kubernetes introducendo costi e complessità.

⁸<https://github.com/google/kctf/blob/v1/docs/security-threat-model.md>

1.3.2 CR Alternativi: CTFBox

CTFBox e' un Cyber Range perfetto per organizzare CTF di tipo Attack/Defense, dove ogni team ha una macchina con uno o piu servizi vulnerabili da difendere, mentre si attaccano i servizi esposti sulle macchine avversarie per recuperare le flag. Durante il gioco, ogni team deve fare attenzione a non rompere i propri servizi, mantenendo un'alta percentuale di SLA. Una SLA bassa comporta una penalita' in termini di punteggio.⁹

Fa uso di **Docker** per gestire i servizi essenziali del gioco (e che costituiscono quindi CTFBox), mentre **Incus** viene utilizzato per gestire l'esecuzione delle macchine virtuali o dei container dei team.

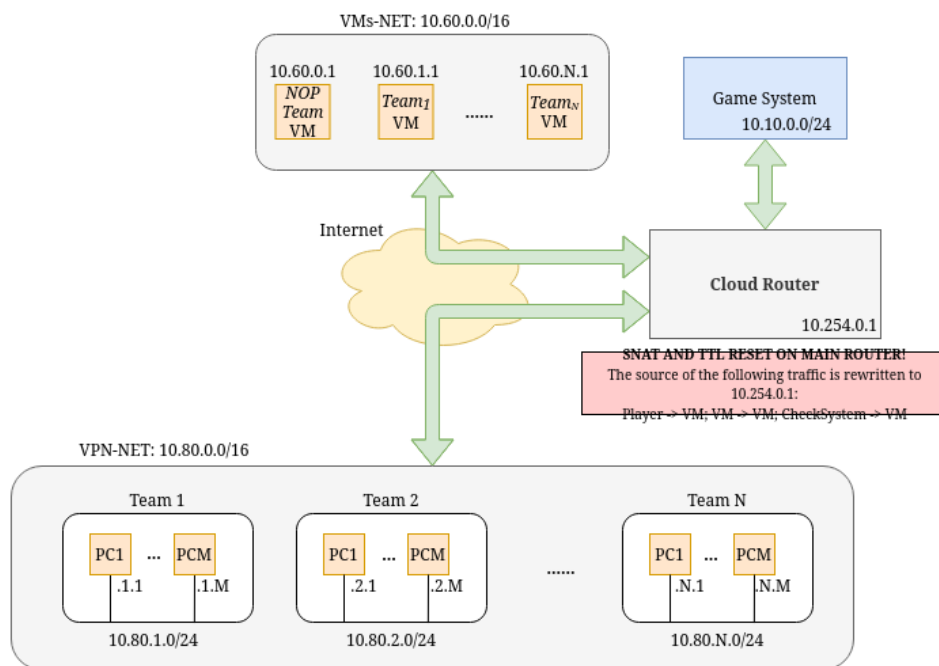


Figura 1.2: Ctfbox Architecture

Router: e' un container Docker, che gestisce le configurazioni della rete WireGuard, permettendo ai giocatori di connettersi alla rete di gioco. Controlla le regole di routing sulla rete, e governa la transizione della rete tra 3 stati diversi:

- **freeze:** in questo stato, nessuno puo' accedere alle macchine vulnerabili
- **lock:** in questo stato, ogni team puo' accedere solo alla propria macchina vulnerabile.
- **unlock:** in questo stato, la rete e' completamente aperta, i team possono attaccare le macchine avversarie.

⁹<https://ctfbox.domy.sh/>

Game System: e' un container Docker, che agisce da cervello centrale per la gestione del gioco.

- Gestisce lo stato della rete (freeze/lock/unlock), comunicando i cambiamenti al router.
- Mantiene il database delle flag, e calcola i punteggi per ogni team.
- Coordina i Checkers, ossia i worker che interrogano i servizi dei team ad ogni tick.

In CTFBox, ad ogni tick (ossia ogni round di gioco), le flag vengono rigenerate su ogni macchina, per ogni servizio vulnerabile, e registrate nel database. L'obiettivo di un team in una competizione Attack/Defense strutturata in questo modo e' di attaccare piu volte lo stesso servizio sulle macchine avversarie, per recuperare quante piu flag possibili, finche le difese avversarie non bloccano gli attacchi.

Checkers, ad ogni tick (o round) fanno due cose:

1. Interrogano tutti i servizi vulnerabili attivi, e determinano la percentuale di SLA per ogni team.
2. Installano le nuove flag sui servizi vulnerabili.

Ogni membro del team, ha la sua configurazione **WireGuard** per conettersi alla propria macchina e per raggiungere le macchine avversarie. Ad ogni giocatore che si connette viene assegnato un'indirizzo nella sottorete `10.80.team_id.0/24` mentre le macchine dei team si trovano nella sottorete `10.60.0.0/16`.

Dunque CTFBox si rileva essere un'ottimo sistema per gestire competizioni Attack/Defense e per regolare l'accesso alle macchine, ma la sua architettura non si applica in contesti in cui vorremmo solo assegnare ad ogni team un insieme di macchine vulnerabili il cui stato verra' poi verificato da un'oracolo.

1.3.3 CR Alternativi: CyTrONE

CyTrONE¹⁰ e' un framework incentrato sul training che integra contemporaneamente sia una piattaforma educativa che un Cyber Range.

La seguente immagine mostra i componenti di CyTrONE:

- **CyLMS**: si occupa d icaricare il contenuto per il training su un Learning Management System come **Moodle**
- **CyRIS** (Cyber Range Instantiation System): si occupa di istanziare l'ambiente di training su cui gli studenti possono condurre opportuni test per avanzare nel training.

¹⁰<https://github.com/crond-jaist/cytrone>

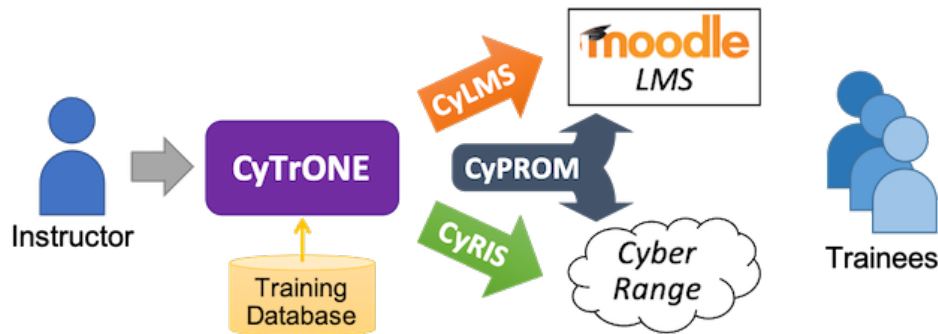


Figura 1.3: Cytrone Architecture

- **CyPROM:** si occupa di tracciare l'avanzamento controllando LMS ed il Cyber Range.

CyTrONE gestisce tre tipi di training¹¹:

- *Training Attack-oriented:* si basa sul capire come riprodurre attacchi a vulnerabilità note.
- *Training Analysis-oriented:* si concentra sull'analisi dell'attacco, per capire come funziona e le sue conseguenze.
- *Training Defense-oriented:* si concentra sulle tecniche di patching delle vulnerabilità, al fine di proteggere il sistema.

Quindi l'esperienza utente si concentra sul fatto che gli studenti possono eseguire un training e visualizzare le risorse associate sul Moodle ed al tempo stesso accedere al Cyber Range associato.

CyTrONE si applica bene facendo le seguenti assunzioni:

- 1) *L'obiettivo di ciascuna attività di formazione può essere formulato come un problema, che può dipendere o meno da altri problemi della stessa lezione (es: trova l'indirizzo IP, trova la flag);*¹¹
- 2) *Il sistema è in grado di stabilire automaticamente se il partecipante abbia risolto il problema, confrontando la risposta fornita con un valore noto*¹¹

L'architettura di CyTrONE è supportata da un *Training Database*, che si occupa di tenere in memoria le informazioni per i vari training, che vengono caricate su CyTrONE da una figura identificata come *Training Coordinator*. Le informazioni mantenute nel database sono per esempio: il training content, file e programmi, macchine virtuali, ecc...¹¹

CyTrONE utilizza un file YAML per definire il contenuto di un training:

```
---
- training:
  - id: L1-EN
    title: |
      Investigate the security of a desktop
      computer
    overview: |
      Today is your first day on the job as
      a sysadmin. Your boss tells you that he
      suspects somebody tried to hack into
      your company's network, and asks you to
      investigate a possible cyber attack
      that may have happened when the system
      administrator was a guy called Daniel
      Craig. The boss sits you in front of
      the previous sysadmin's computer, and
      wishes you good luck.

      You glance at the machine and
      reluctantly get to work.
    level: 1

  questions:
    - id: L1-EN-001
      body: |
        The operating system and kernel
        release number can tell you about
        the possible vulnerabilities of a
        computer. Find out the full kernel
        release number of the machine (e.g.,
        3.4.5-6.7.8.abc.x86_64).
      answer: 3.10.0-514.21.1.el7.x86_64
      hints:
        - You can use the command uname to
          find out OS details.
        - $ uname -r
        - An alternative solution is to check
          the /proc/version file.
```

Figura 1.4: Cytrone Training

```
---
- host_settings:
  - id: host_1
    mgmt_addr: {{ host_mgmt_addr }}
    virbr_addr: {{ host_virbr_addr }}
    account: {{ host_account }}

- guest_settings:
  - id: desktop
    basevm_host: host_1
    basevm_config_file: /imgs/basevm_desktop.xml
    basevm_type: kvm
    basevm_name: basevm_desktop
    tasks:
      - add_account:
          - account: daniel
            passwd: daniel_passwd
            full_name: Daniel Craig
      - install_package:
          - package_manager: yum
            name: wireshark
      - emulate_attack:
          - attack_type: ssh_attack
            target_account: daniel
            attempt_number: 123
            attack_time: 20170123
      - emulate_malware:
          - name: DAEMON
            cpu_utilization: 40
            mode: dummy_calculation

- clone_settings:
  - range_id: {{ clone_range_id }}
    hosts:
      - host_id: host_1
        instance_number: {{ clone_instance_number }}
        guests:
          - guest_id: desktop
            number 1
            entry_point: yes
        topology:
          - type: custom
            networks:
              - name: office
                members: desktop.eth0
```

Figura 1.5: Cytrone Cyberrange

E similmente, si usa un file YAML per definire la specifica del Cyber Range:

A partire dalla configurazione YAML del Cyber Range, il sotto-sistema CyRIS si occupa di istanziarlo. Successivamente, in caso di un training **Defense-oriented** l'utente richiede l'attacco associato alla vulnerabilit  all'interno del sistema e ne viene registrato l'esito.

Tutta via, sebbene il sistema sia ben ingegnerizzato soffre di alcuni problemi:

1. Il deploy delle vulnerabilit  avviene su macchine virtuali, che possono costituire un alto overhead in termini di risorse, ed tempi di approvvigionamento lunghi per l'infrastruttura.
2. Il payload viene inviato remotamente dal `cyberattack controller` (che non fa altro che eseguire `Metasploit`). Tutta via il traffico di rete puo' essere intercettato dallo studente, ottenendo cos  suggerimenti per la soluzione.
3. L'esecuzione del payload puo scatenare la produzione di file log accessibili allo studente.
4. Necessita di un LMS (per design) per tenere traccia del training progress.

TODO: analisi del grafico!

¹¹<https://www.sciencedirect.com/science/article/pii/S0167404818306527>

Elenco delle figure

1.1	Kctf Architecture	5
1.2	Ctfbox Architecture	8
1.3	Cytrone Architecture	10
1.4	Cytrone Training	11
1.5	Cytrone Cyberrange	12

Elenco dei codici

Elenco degli algoritmi